CSE 4309: Theory of Computing

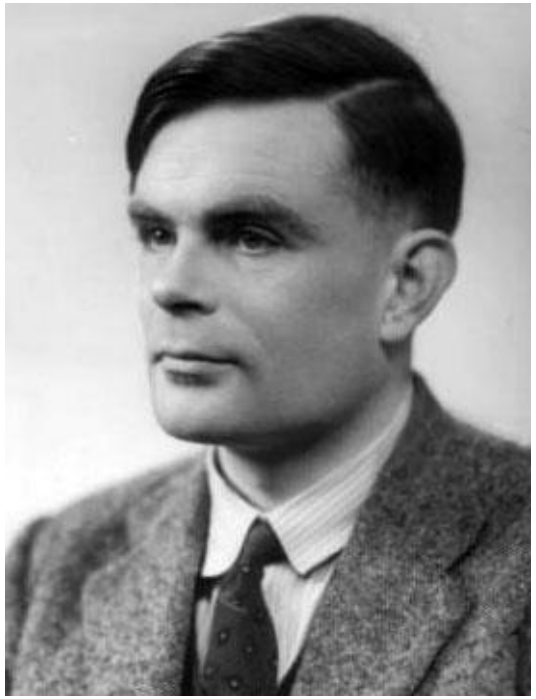
Lecture 5 — Turing Machines

Syed Rifat Raiyan
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh
Email: rifatraiyan@iut-dhaka.edu

Lecture Plan

The agenda for today

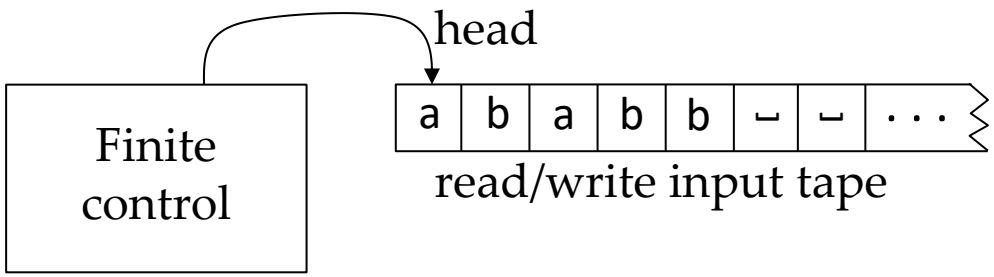
- What is a Turing Machine (TM)?
- T-recognizable and T-decidable languages
- Equivalence of variants of TM models
- Church-Turing Thesis
- Problem-to-String Encoding
- The Halting Problem



Alan Turing
1912–1954

Turing Machine

What is it?



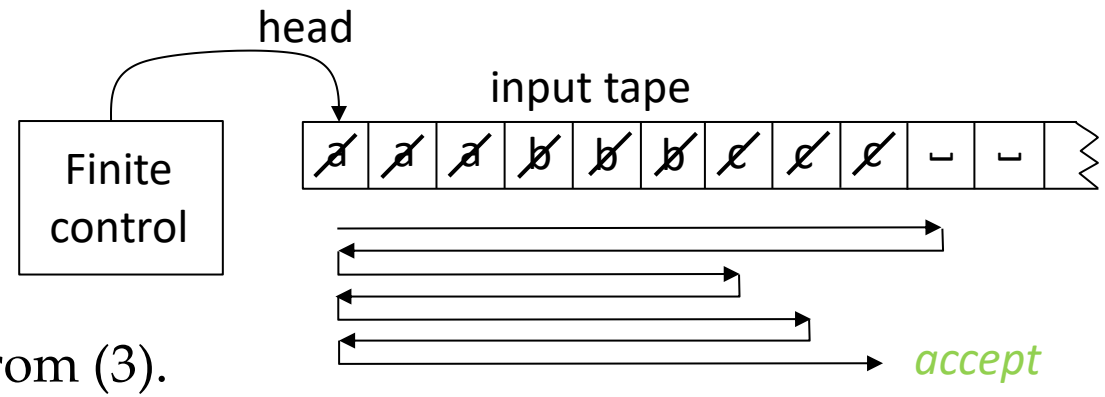
1) Head can **read** and **write**
2) Head is two way (can move left or right)
3) Tape is infinite (to the right)
4) Infinitely many blanks "␣" follow input
5) Can accept or reject any time (not only at end of input)

Turing Machine

How does it work?

TM recognizing $B = \{a^k b^k c^k \mid k \geq 0\}$

1) Scan right until ␣ while checking if input is in $a^* b^* c^*$, *reject* if not.
2) Return head to left end.
3) Scan right, crossing off single a, b, and c.
4) If the last one of each symbol, *accept*.
5) If the last one of some symbol but not others, *reject*.
6) If all symbols remain, return to left end and repeat from (3).



Turing Machine

The formal definition

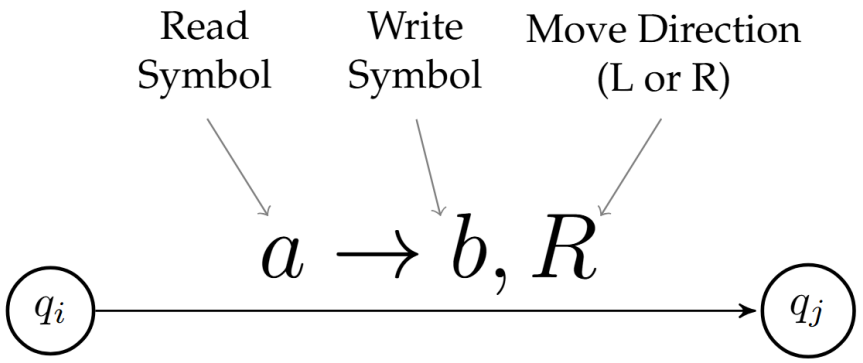
Defn: A <u>Turing Machine</u> (TM) is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$

Σ input alphabet

Γ tape alphabet $(\Sigma \subseteq \Gamma)$

δ: $Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ (L = Left, R = Right)

$\delta(q, a) = (r, b, R)$



On input w a TM M may halt (enter q_{acc} or q_{rej}) or M may run forever ("loop").

So M has 3 possible outcomes for each input w:

1. *Accept* w (enter q_{acc})
2. *Reject* w by halting (enter q_{rej})
3. *Reject* w by looping (running forever)

This Turing machine model is deterministic.
How would we change it to be nondeterministic?

Change δ to be δ: $Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$

TM Recognizers and Deciders

What are they?

Let M be a TM. Then $L(M) = \{w \mid M \text{ accepts } w\}$.

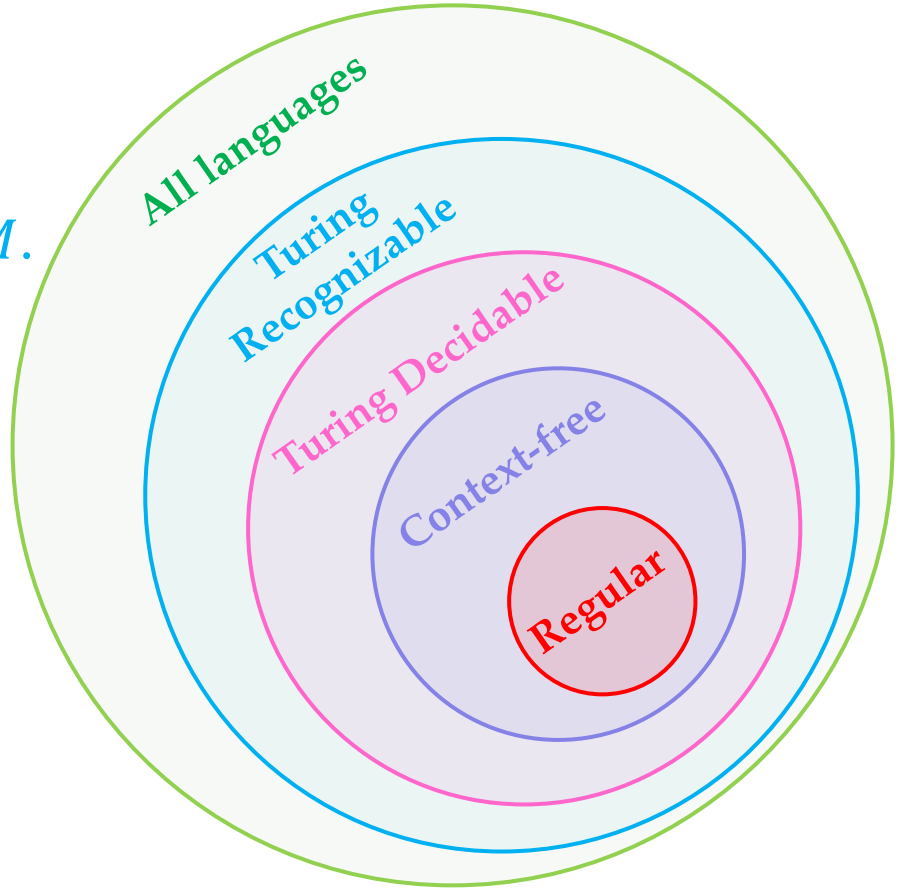
Say that M recognizes A if $A = L(M)$.

Defn: A is Turing-recognizable if $A = L(M)$ for some TM M.

Defn: TM M is a decider if M halts on all inputs.

Say that M decides A if $A = L(M)$ and M is a decider.

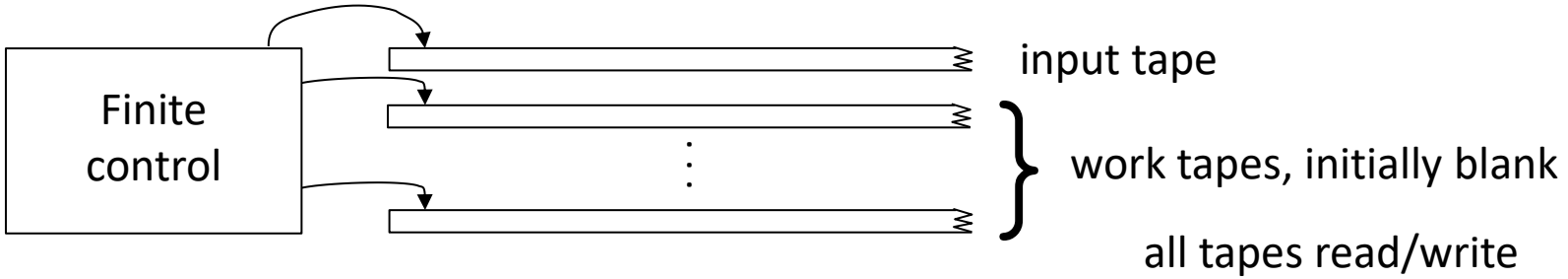
Defn: A is Turing-decidable if $A = L(M)$ for some TM decider M.



Chomsky Hierarchy

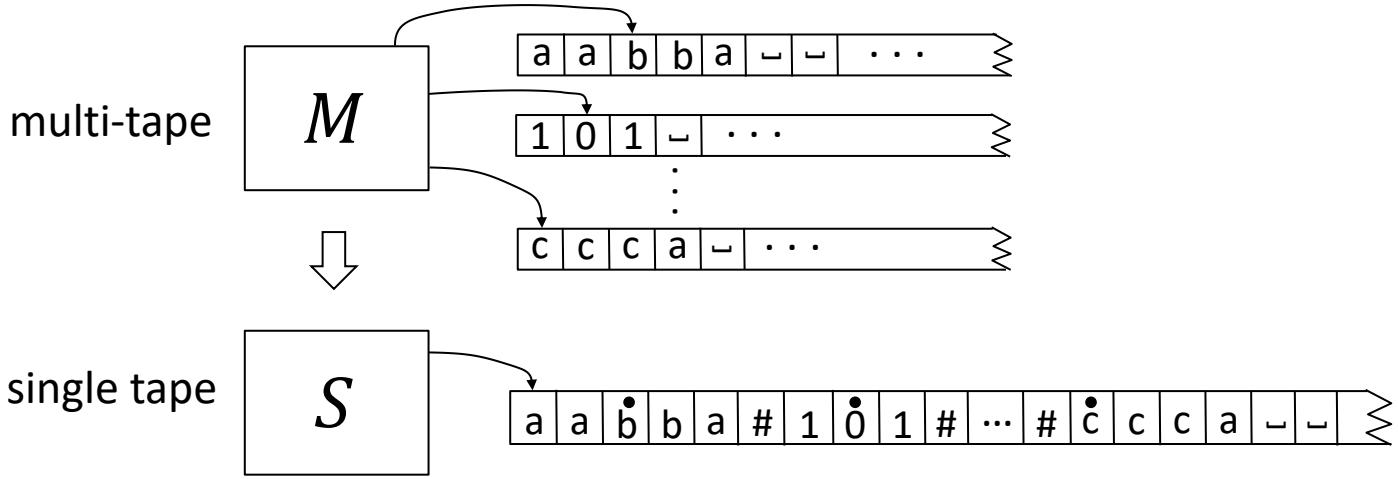
Multi-tape Turing Machines

How are they equivalent to Single-tape TM?



Theorem: A is T-recognizable iff some multi-tape TM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert multi-tape to single tape:



S simulates M by storing the contents of multiple tapes on a single tape in "blocks". Record head positions with dotted symbols.

Some details of S:

1) To simulate each of M's steps
   a. Scan entire tape to find dotted symbols.
   b. Scan again to update according to M's δ.
   c. Shift to add room as needed.
2) Accept/reject if M does.

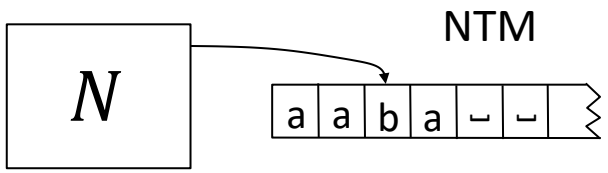
Nondeterministic Turing Machines

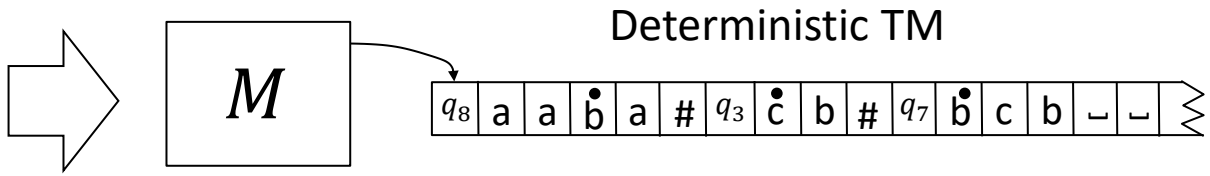
What are they?

A <u>Nondeterministic TM</u> (NTM) is similar to a Deterministic TM except for its transition function δ: $Q \times \Gamma \to \mathcal{P}(\, Q \times \Gamma \times \{L, R\}\,)$.

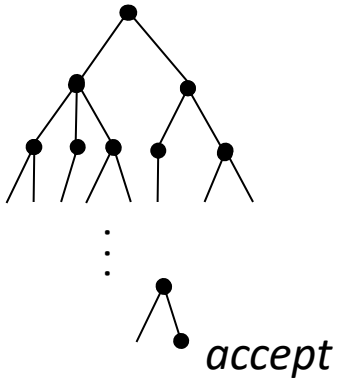
Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\to)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.





Nondeterministic computation tree for N on input w.



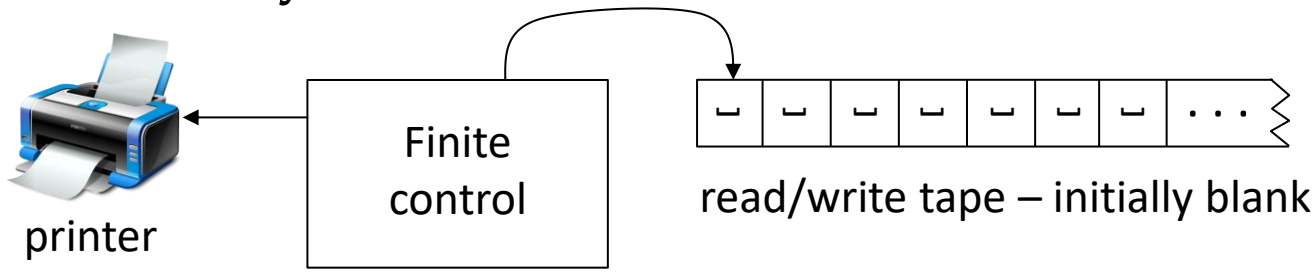
M simulates N by storing each thread's tape in a separate "block" on its tape.
Also need to store the head location, and the state for each thread, in the block.

If a thread forks, then M copies the block.

If a thread accepts then M accepts.

Turing Enumerators

What are they?



Defn: A <u>Turing Enumerator</u> is a deterministic TM with a printer.

It starts on a blank tape and it can print strings $w_1, w_2, w_3, \ldots$ possibly going forever.

Its language is the set of all strings it prints. It is a generator, not a recognizer.

For enumerator E we say $L(E) = \{w \mid E \text{ prints } w\}$.

Theorem: A is T-recognizable iff $A = L(E)$ for some T-enumerator E.

Proof: $(\leftarrow)$ Convert E to equivalent TM M.

$M =$ for input w:

Simulate E (on blank input).

Whenever E prints x, test $x = w$.

Accept if $=$ and continue otherwise.

Proof: $(\rightarrow)$ Convert TM M to equivalent enumerator E.

$E =$ Simulate M on each w_i in $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, \ldots\}$

If M accepts w_i then print w_i .

Continue with next w_i .

Problem: What if M on w_i loops?

Fix: Simulate M on $w_1, w_2, \ldots, w_i$ for i steps, for $i = 1, 2, \ldots$

Print those w_i which are accepted.

Church–Turing Thesis

What is it?

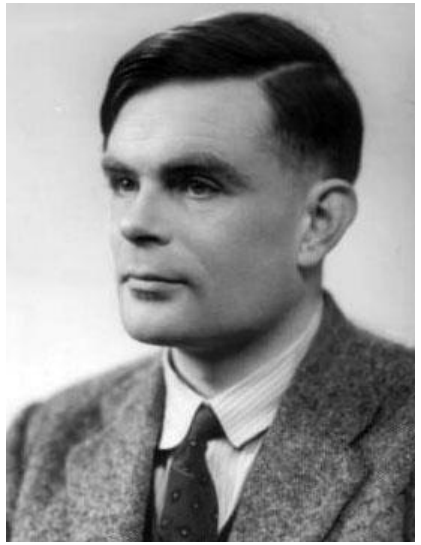
It is a thesis about the nature of *computable* functions.

It states that a function on the natural numbers can be calculated by an *effective* method if and only if it is computable by a Turing machine.

Finite-time
+
Deterministic
procedure

Alonzo Church
1903–1995

Algorithm	=	Turing machine
Intuitive		Formal



Alan Turing
1912–1954

Instead of Turing machines,
can use any other "reasonable" model
of unrestricted computation:
λ-calculus, random access machine, your
favorite programming language, …

Encodings and TMs

What are the notations?

Notation for encoding objects into strings

- If O is some object (e.g., polynomial, automaton, graph, etc.), we write $\langle O \rangle$ to be an encoding of that object into a string.

- If $O_1, O_2, \ldots, O_k$ is a list of objects then we write $\langle O_1, O_2, \ldots, O_k \rangle$ to be an encoding of them together into a single string.

Notation for writing Turing machines

We will use high-level English descriptions of algorithms when we describe TMs, knowing that we could (in principle) convert those descriptions into states, transition function, etc. Our notation for writing a TM M is

$M =$ "On input w

[English description of the algorithm]"

Turing Machine

Revisiting an example

TM M recognizing $B = \{\mathrm{a}^k \mathrm{b}^k \mathrm{c}^k \mid k \geq 0\}$

M = "On input w

1. Check if $w \in \mathrm{a}^* \mathrm{b}^* \mathrm{c}^*$, *reject* if not.
2. Count the number of a's, b's, and c's in w.
3. *Accept* if all counts are equal; *reject* if not."

High-level description is ok.

You don't necessarily need to manage tapes, states, etc…

Turing Machine

Left-end of the tape

Problem

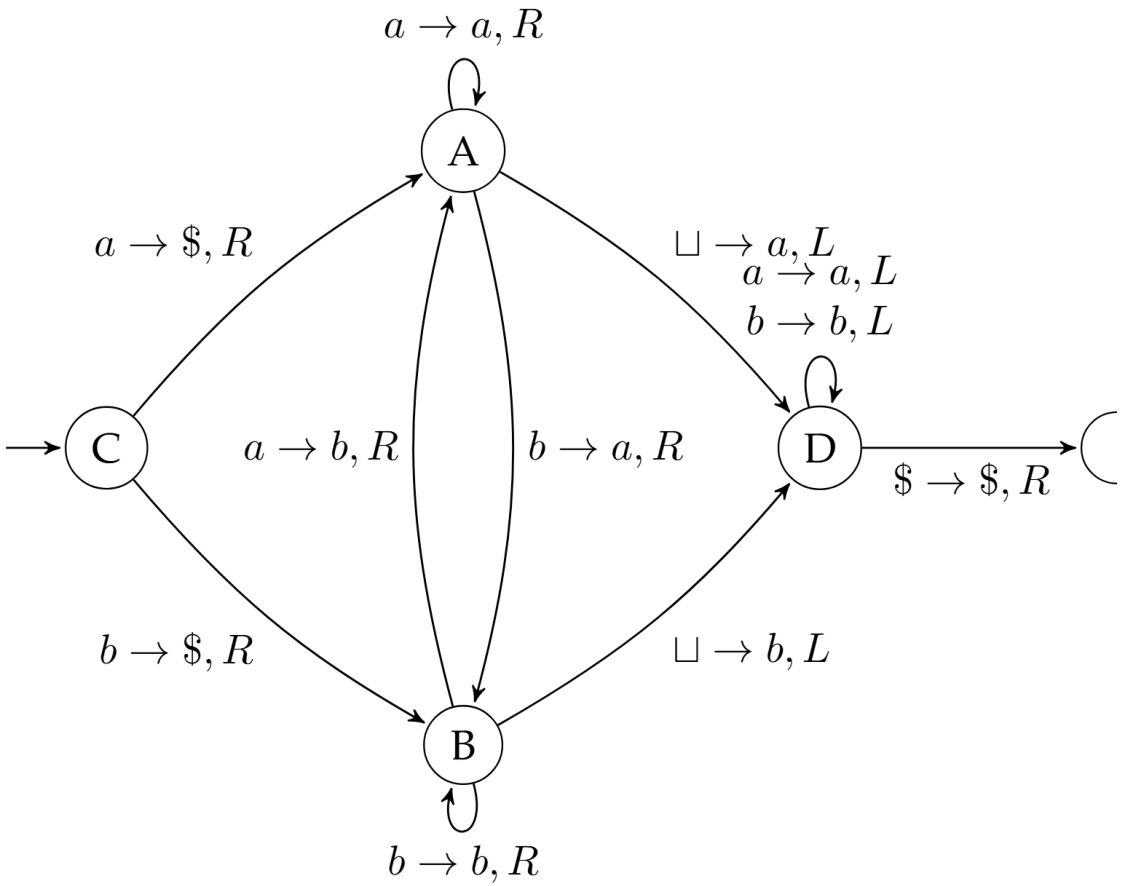
How can we recognize the left end of the tape?

Goal: Place a special symbol \$ on the left end of the tape and shift the input one cell to the right.

$$\underline{a} \quad a \quad a \quad b \quad b \quad a \quad \sqcup \quad \cdots$$

$$\Downarrow$$

$$\$ \quad \underline{a} \quad a \quad a \quad b \quad b \quad a \quad \sqcup \cdots$$

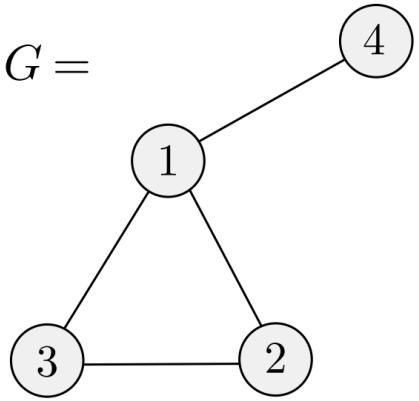


Turing Machine

An example

$$A = \{\langle G \rangle \mid G \text{ is a connected undirected graph}\}$$

$G =$



$\langle G \rangle =$

`(1,2,3,4)((1,2),(2,3),(3,1),(1,4))`

The following is a high-level description of a TM M that decides A.

$M =$ "On input $\langle G \rangle$, the encoding of a graph G:

1. Select the first node of G and mark it.
2. Repeat the following stage until no new nodes are marked:
3. For each node in G, mark it if it is attached by an edge to a node that is already marked.
4. Scan all the nodes of G to determine whether they all are marked. If they are, *accept*; otherwise, *reject*."

Turing Machine

Decidability (*Acceptance problem*)

Let $A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA and } B \text{ accepts } w\}$

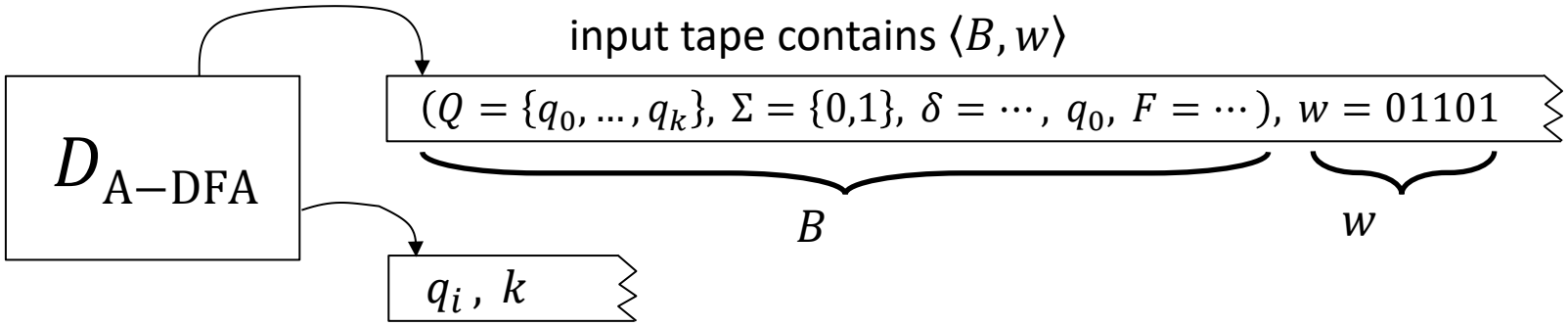
Theorem: A_{DFA} is decidable

Proof: Give TM $D_{\text{A-DFA}}$ that decides A_{DFA}.

$D_{\text{A-DFA}}$ = "On input s

1. Check that s has the form $\langle B, w\rangle$ where B is a DFA and w is a string; *reject* if not.
2. Simulate the computation of B on w.
3. If B ends in an accept state then *accept*. If not then *reject*."

Shorthand:
On input $\langle B, w\rangle$



Turing Machine

Decidability (*Acceptance problem*)

Let $A_{\text{NFA}} = \{\langle B, w\rangle |\ B \text{ is a NFA and } B \text{ accepts } w\}$

Theorem: A_{NFA} is decidable

Proof: Give TM $D_{\text{A-NFA}}$ that decides A_{NFA}.

$D_{\text{A-NFA}} =$ "On input $\langle B, w\rangle$

1. Convert NFA B to equivalent DFA B'.
2. Run TM $D_{\text{A-DFA}}$ on input $\langle B', w\rangle$. [Recall that $D_{\text{A-DFA}}$ decides A_{DFA}]
3. *Accept* if $D_{\text{A-DFA}}$ accepts.
   Reject if not."

New element: Use conversion construction and previously constructed TM as a subroutine.

Turing Machine

Decidability (*Emptiness problem*)

Let $E_{\text{DFA}} = \{\langle B \rangle \mid B \text{ is a DFA and } L(B) = \emptyset\}$

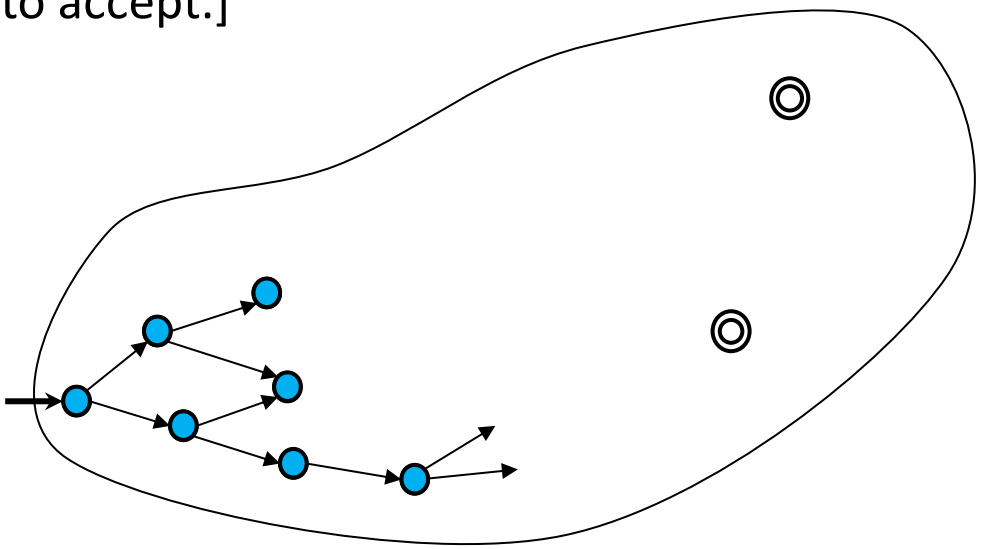
Theorem: E_{DFA} is decidable

Proof: Give TM $D_{\text{E-DFA}}$ that decides E_{DFA} .

$D_{\text{E-DFA}}$ = "On input $\langle B \rangle$ [IDEA: Check for a path from start to accept.]

1. Mark start state.
2. Repeat until no new state is marked:

   Mark every state that has an incoming arrow from a previously marked state.
3. *Accept* if no accept state is marked.
   Reject if some accept state is marked."



Turing Machine

Decidability (*Equivalence problem*)

Let $EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$

Theorem: EQ_{DFA} is decidable

Proof: Give TM $D_{\text{EQ-DFA}}$ that decides EQ_{DFA} .

$D_{\text{EQ-DFA}}$ = "On input $\langle A, B\rangle$ [IDEA: Make DFA C that accepts w where A and B

1. Construct DFA C where $L(C) = \left(L(A) \cap \overline{L(B)}\right) \cup \left(\overline{L(A)} \cap L(B)\right)$.
2. Run $D_{\text{E-DFA}}$ on $\langle C\rangle$.
3. *Accept* if $D_{\text{E-DFA}}$ accepts.
   Reject if $D_{\text{E-DFA}}$ rejects."

Turing Machine

Decidability (*Acceptance problem*)

Let $A_{\text{CFG}} = \{\langle G, w\rangle \mid G \text{ is a CFG and } w \in L(G)\}$

Theorem: A_{CFG} is decidable

Proof: Give TM $D_{\text{A-CFG}}$ that decides A_{CFG}.

$D_{\text{A-CFG}}$ = "On input $\langle G, w\rangle$

1. Convert G into CNF.
2. Try all derivations of length $2|w| - 1$.
3. *Accept* if any generate w.
   Reject if not.

Corollary: Every CFL is decidable.

Proof: Let A be a CFL, generated by CFG G.

Construct TM M_G = "on input w

1. Run $D_{\text{A-CFG}}$ on $\langle G, w\rangle$.
2. *Accept* if $D_{\text{A-CFG}}$ accepts
   Reject if it rejects."

Self-study:

Emptiness problem and

Equivalence problem for CFGs

Recall Chomsky Normal Form (CNF) only allows rules:

A → BC

B → b

Lemma 1: Can convert every CFG into CNF.

Proof and construction in book.

Lemma 2: If H is in CNF and $w \in L(H)$ then every derivation of w has $2|w| - 1$ steps.

Proof: exercise.

Turing Machine

Decidability (*Acceptance problem*)

Let $A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$

Theorem: A_{TM} is not decidable

Proof: See **Theorem 4.11** from the book. (*Self-study*)

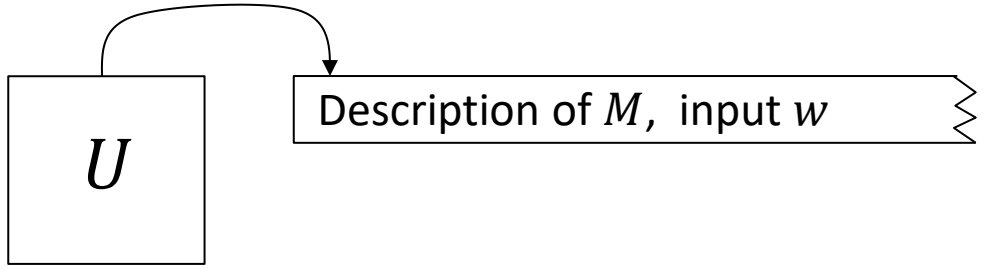
Theorem: A_{TM} is T-recognizable

Proof: The following TM U recognizes A_{TM}

U = "On input $\langle M, w\rangle$

1. Simulate M on input w.
2. *Accept* if M halts and accepts.
3. *Reject* if M halts and rejects.
4. ~~*Reject* if M never halts~~." Not a legal TM action.

Turing's original "Universal Computing Machine"



Von Neumann said U inspired the concept of a stored program computer.

Turing Machine

Reducibility (*Halting problem*)

Use our knowledge that A_{TM} is undecidable to show other problems are undecidable.

Defn: $HALT_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ halts on input } w\}$

Theorem: $HALT_{\mathrm{TM}}$ is undecidable
Proof by contradiction, showing that A_{TM} is reducible to $HALT_{\mathrm{TM}}$:

Assume that $HALT_{\mathrm{TM}}$ is decidable and show that A_{TM} is decidable (false!).
Let TM R decide $HALT_{\mathrm{TM}}$.
Construct TM S deciding A_{TM}.

$S =$ "On input $\langle M, w\rangle$

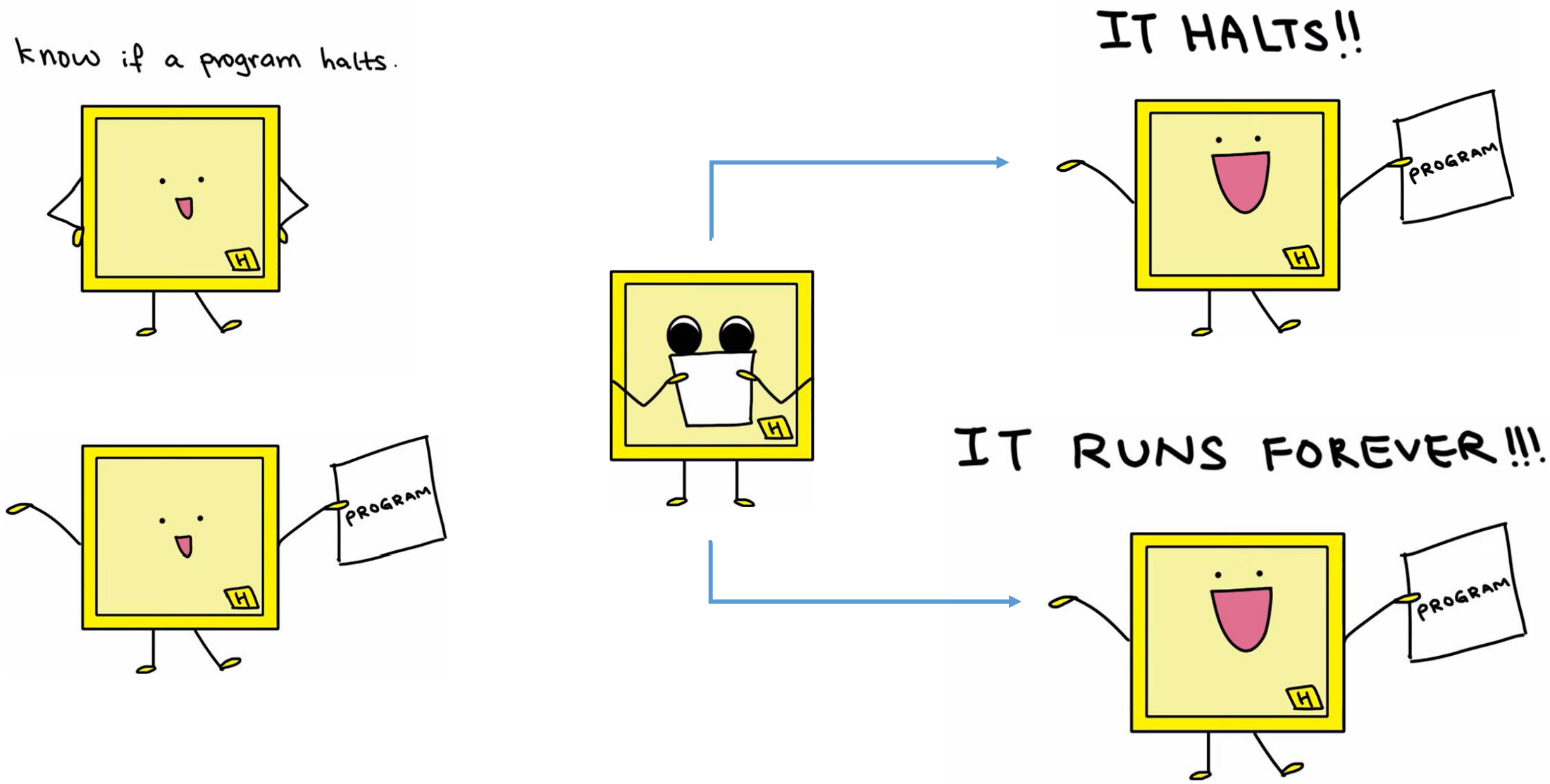
1. Use R to test if M on w halts. If not, reject.
2. Simulate M on w until it halts (as guaranteed by R).
3. If M has accepted then *accept*.
   If M has rejected then *reject*.

TM S decides A_{TM}, a contradiction. Therefore $HALT_{\mathrm{TM}}$ is undecidable.

Formal Proof in **Theorem 5.1**

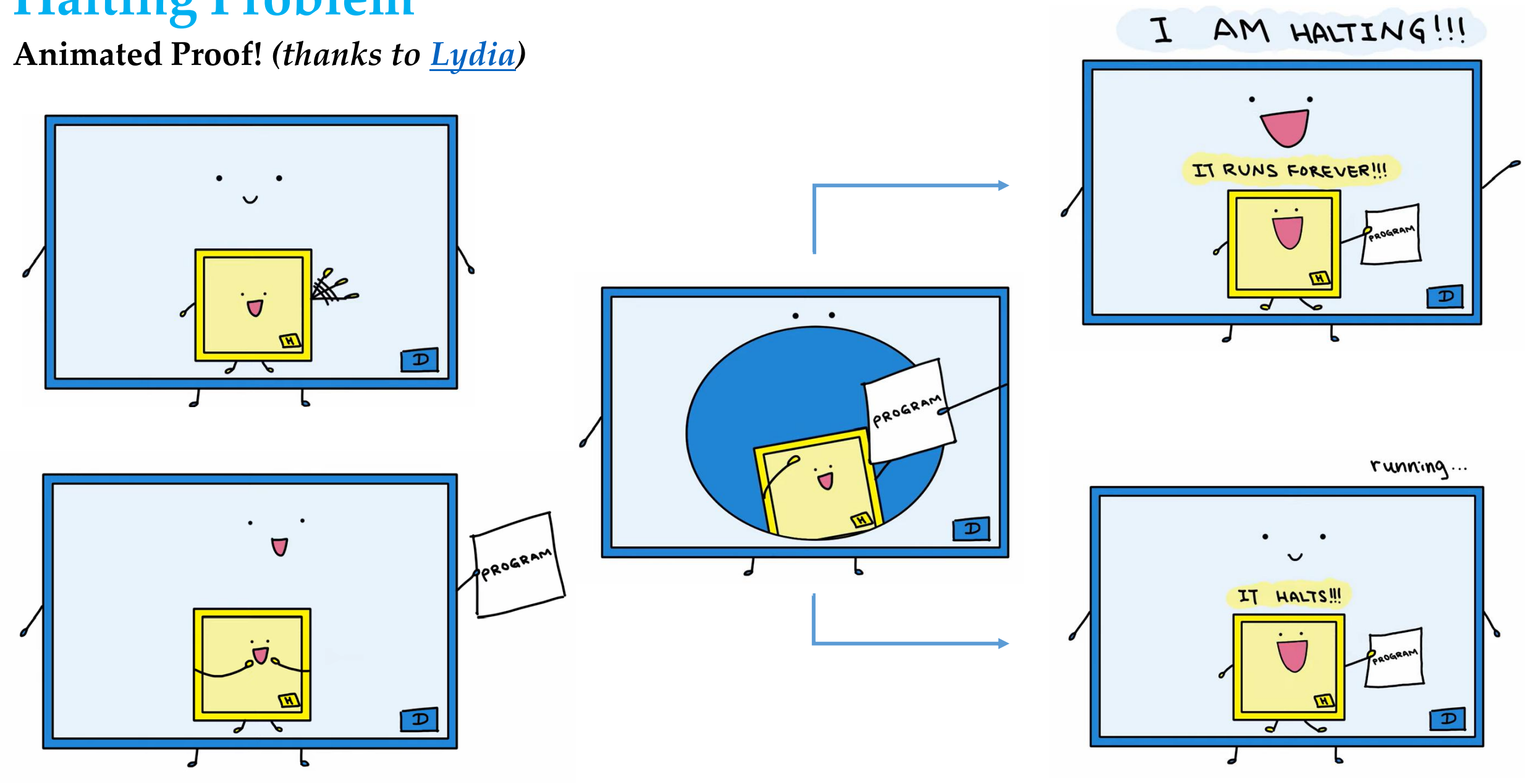
Halting Problem

Animated Proof! ***(thanks to Lydia)***



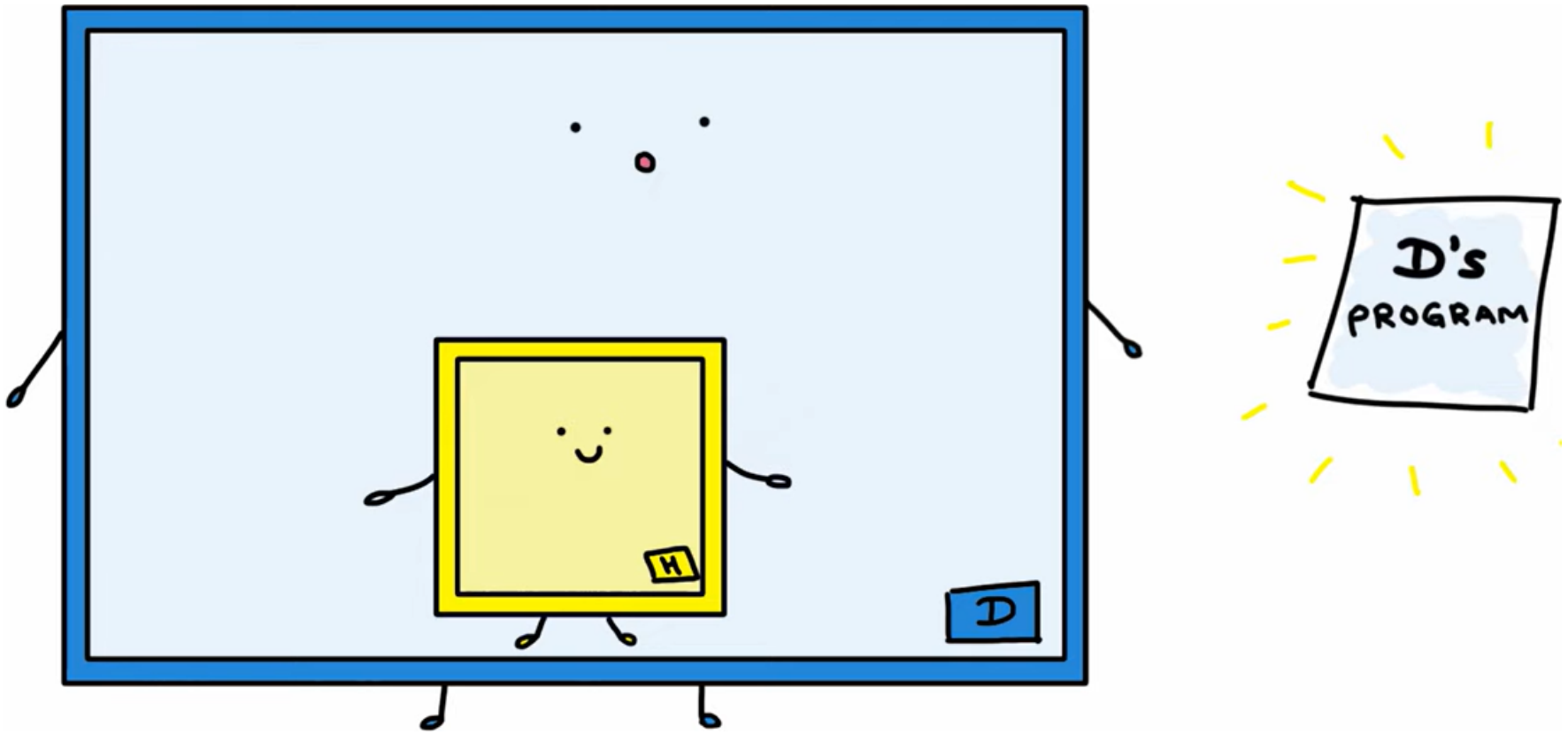
Halting Problem

Animated Proof! *(thanks to Lydia)*



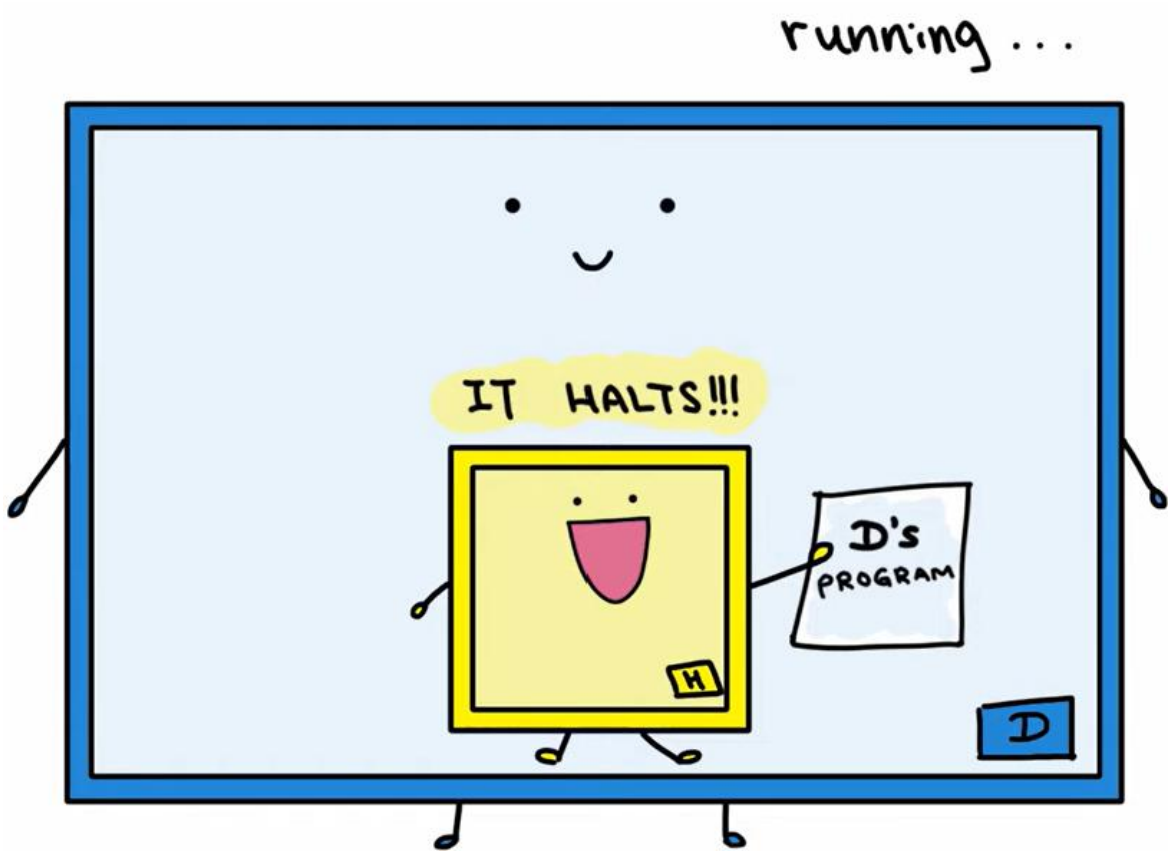
Halting Problem

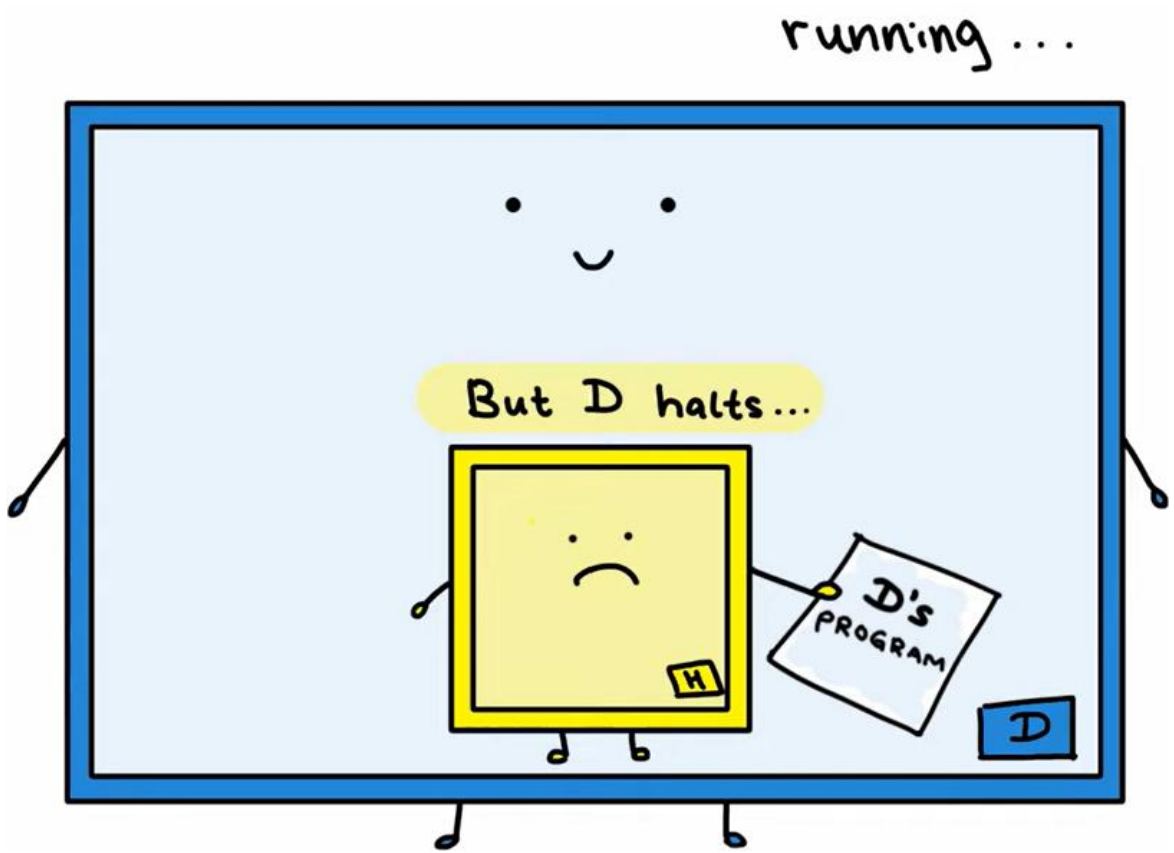
Animated Proof! ***(thanks to Lydia)***



Halting Problem

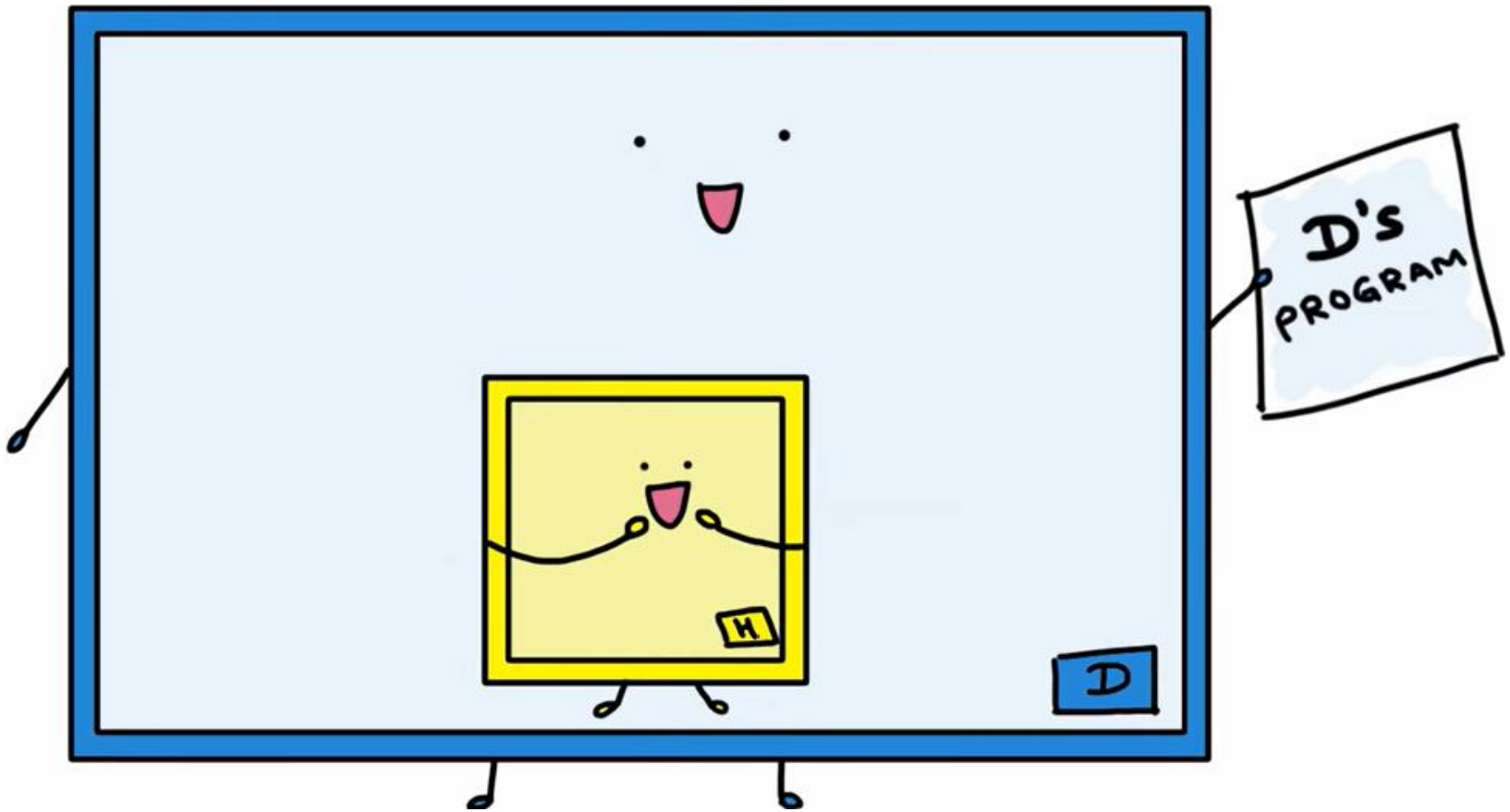
Animated Proof! ***(thanks to Lydia)***





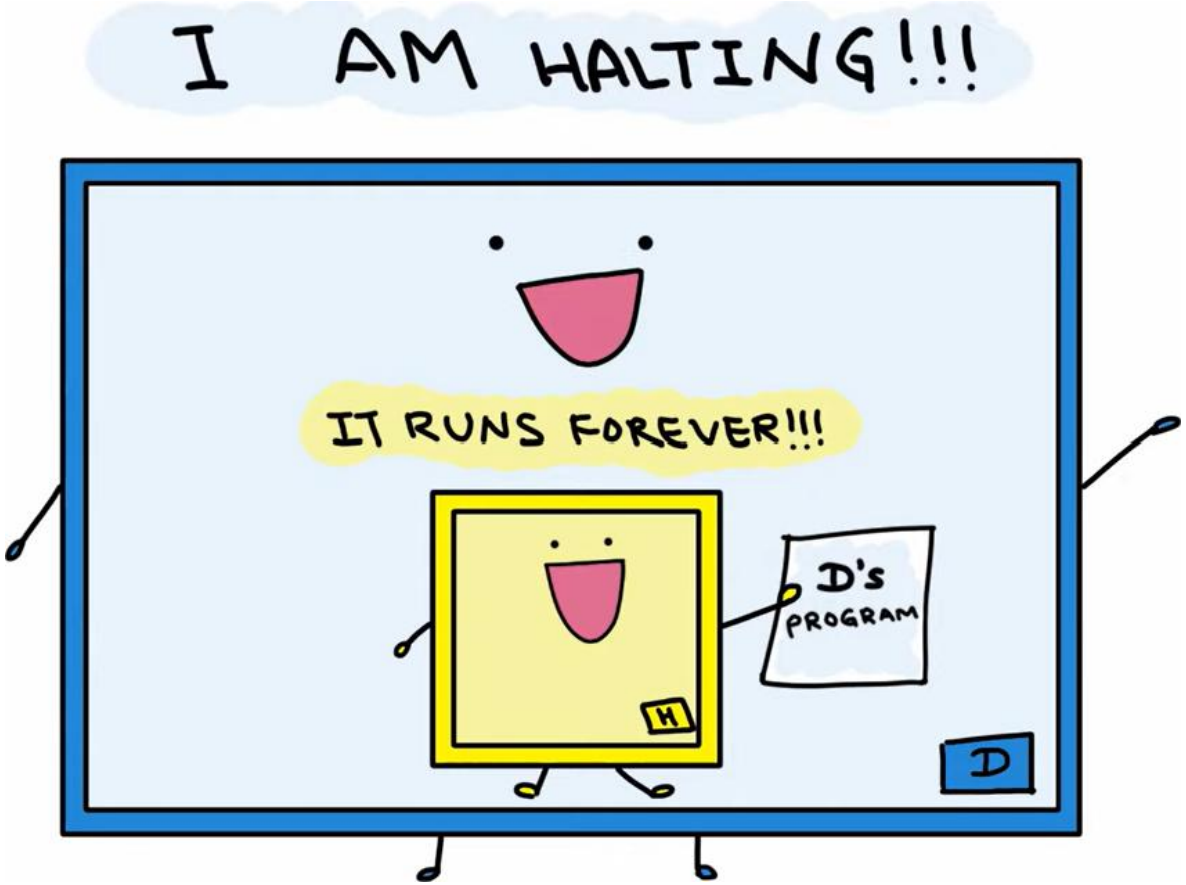
Halting Problem

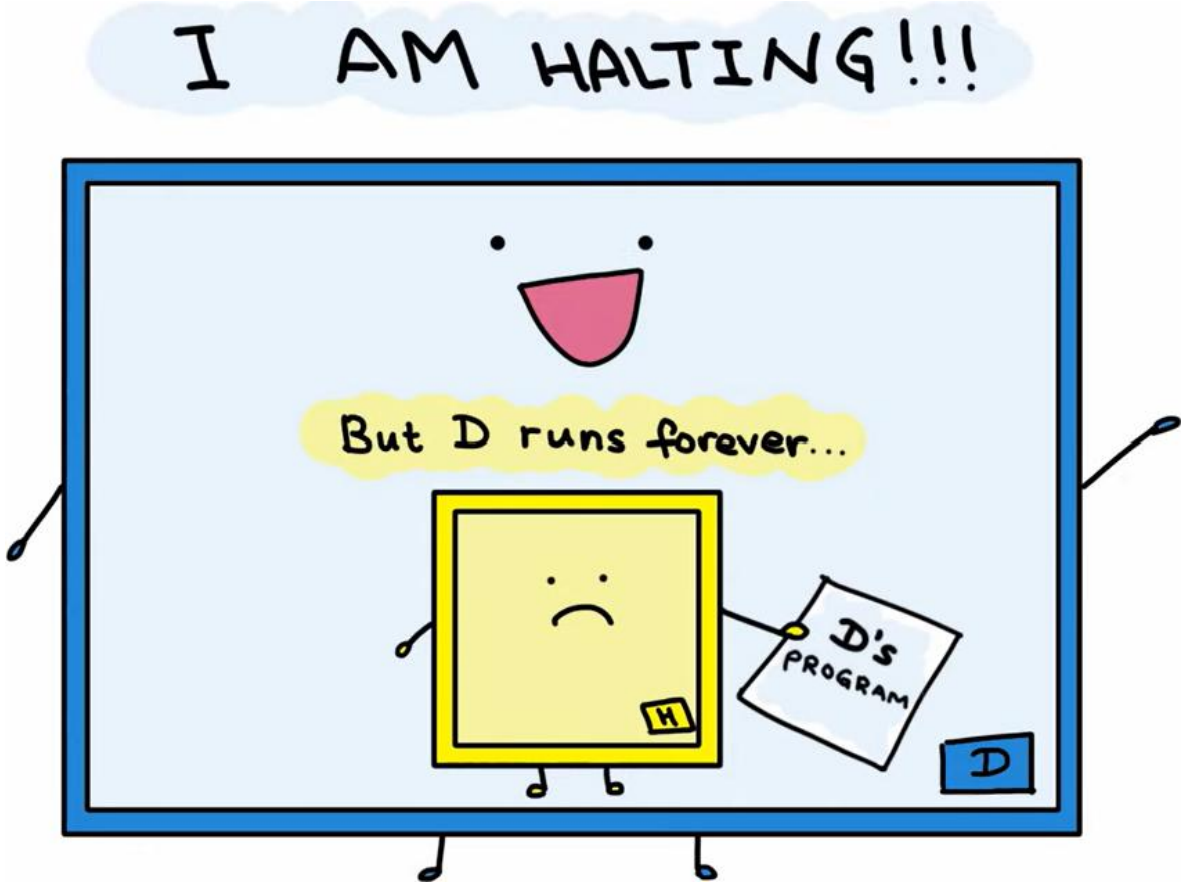
Animated Proof! ***(thanks to Lydia)***



Halting Problem

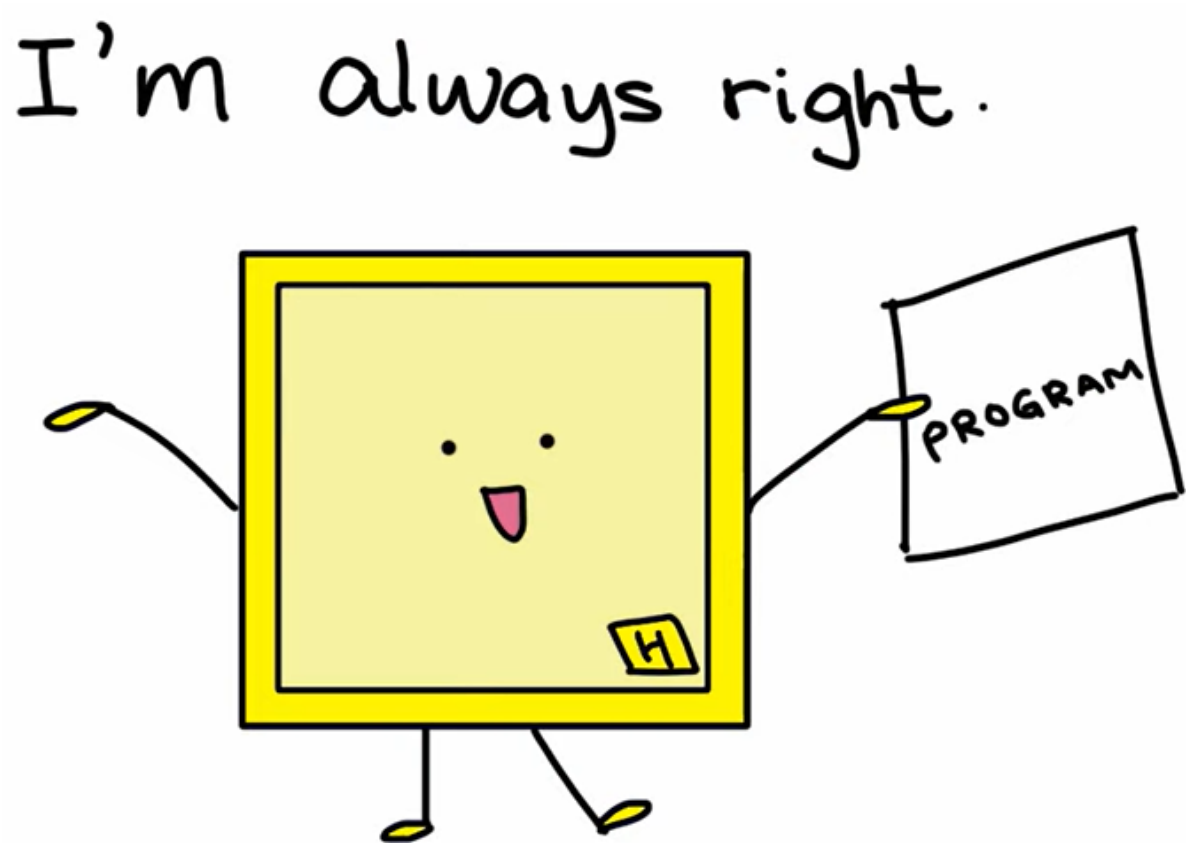
Animated Proof! *(thanks to Lydia)*





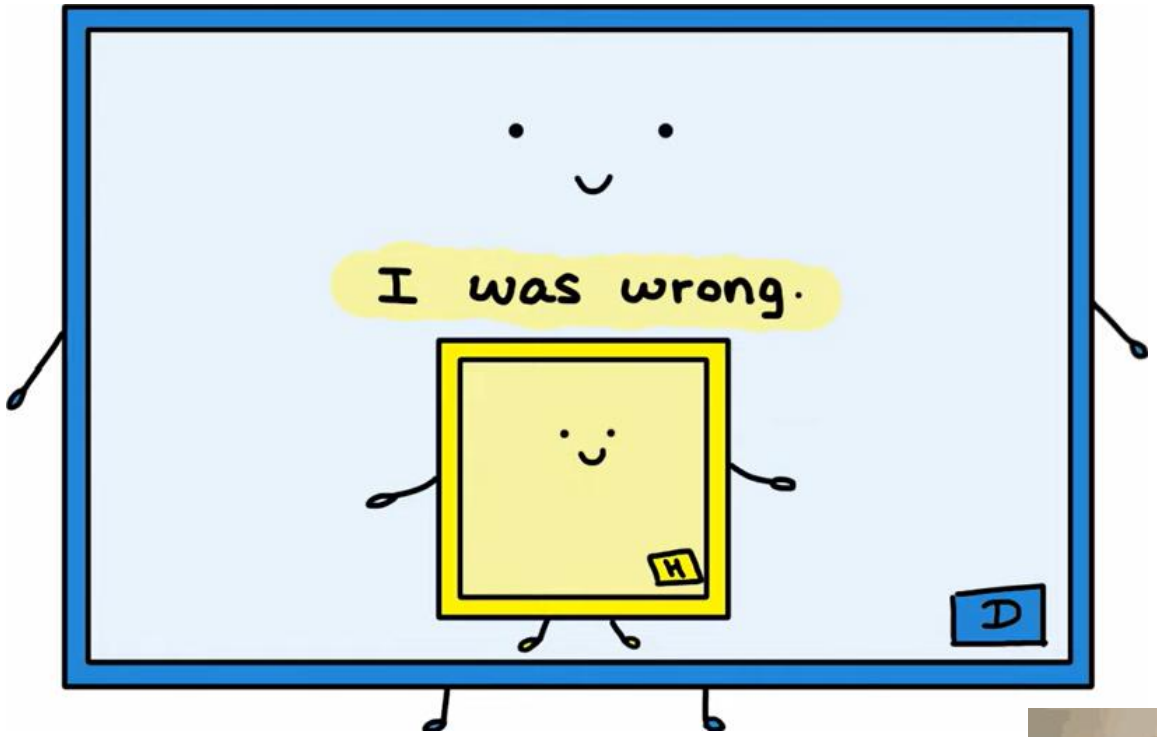
Halting Problem

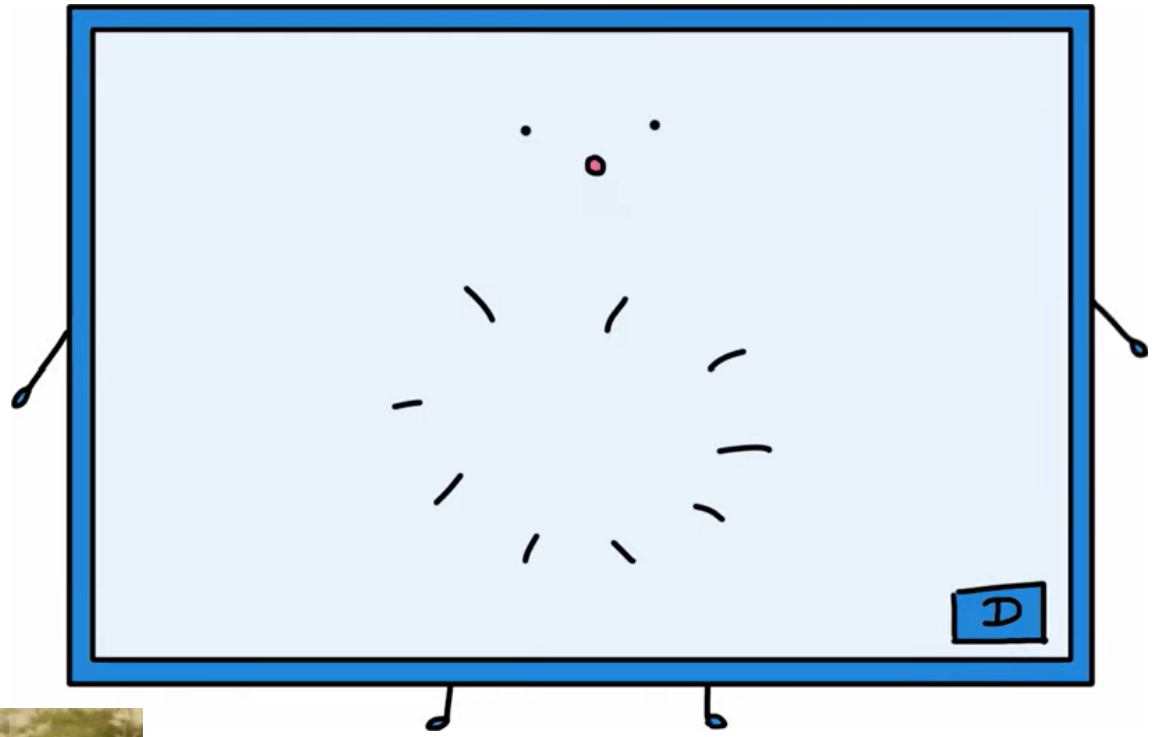
Animated Proof! *(thanks to Lydia)*



Halting Problem

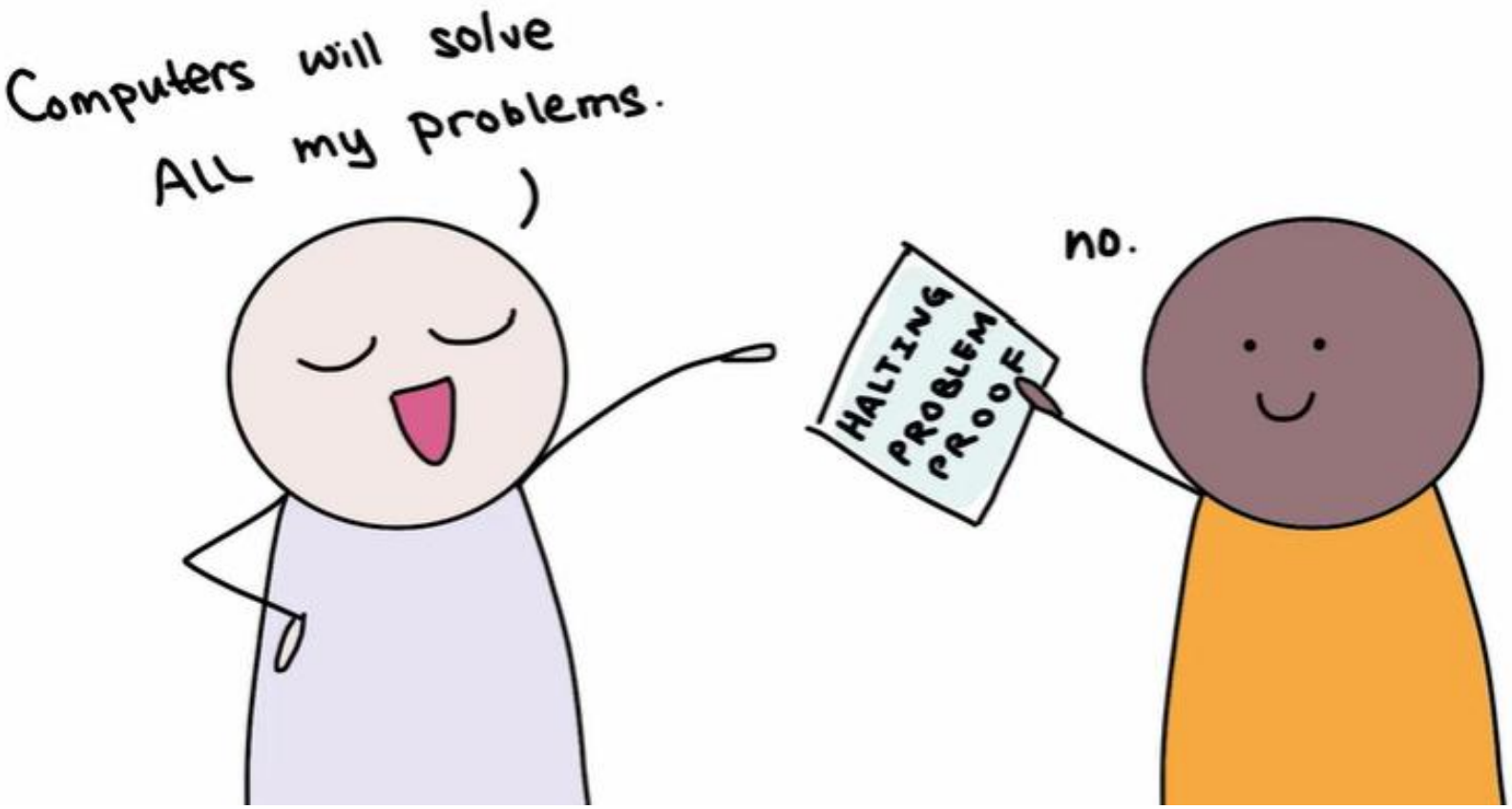
Animated Proof! *(thanks to Lydia)*





Halting Problem

Animated Proof! ***(thanks to Lydia)***



Not all problems are solvable!